MIT 8103 Advanced Database Systems

Individual portfolio summary

Student: Asiya Ya'u
Registration number: 2026/A/MIT/0763
Programme: Master of Information Technology, Miva Open University
Session: 2026/2027, First Semester
Case study: Amana Community Clinic (simulated)

This note covers the five practical activities and the reflection. The same case study is used throughout.

GitHub profile: https://github.com/Asiya-Ya-u
Repository: https://github.com/Asiya-Ya-u/advanced-database-portfolio (branch `master`)

The organisation

Amana Community Clinic is a simulated primary-care clinic in Kano, with General Outpatient, Maternal Health, Paediatrics, a laboratory, and a pharmacy. The users are reception, doctors, pharmacy, the laboratory, and accounts. The database registers patients, books one doctor for one minute, records the visit and a primary diagnosis, dispenses medicines, orders tests, and takes payment. The loaded sample has 120 patients, 428 appointments, 284 encounters, 252 invoices, and 194 payments.

Portfolio 1: design

The relational schema is in third normal form. A repeating visit sheet, with several drugs in one cell and the patient's phone repeated on every row, was split into patient, staff, department, appointment, encounter, diagnosis, prescription, prescription item, medication, stock movement, laboratory test and order, invoice, invoice line, and payment. Folder numbers, staff licences, and receipt numbers are unique. One booking cannot occupy the same doctor in the same minute. Stock cannot become negative. The ER diagram is `portfolio-1-database-design/er-diagram.svg`.

Portfolio 2: queries and plans

Ten queries were planned with `EXPLAIN QUERY PLAN` before and after indexing. A name lookup changed from a scan of `patient` to a search on `idx_patient_name`. Open invoices (58 in the sample) are now found from `idx_invoice_status`, and their lines from `idx_invoice_line_invoice`. A patient's history and a doctor's workload use indexes on `encounter`. The laboratory backlog uses `(status, ordered_at)`.

Some indexes were refused. The doctor day-list was already served by the unique constraint on doctor and time, so a second index was not added. An index on `stock_qty` does not serve `stock_qty <= reorder_level`, and with 12 medicines the scan is the right plan. Monthly revenue must read every payment, so it still scans that table. September receipts were NGN 237,025.00, October NGN 232,410.00, and November NGN 185,350.00.

Portfolio 3: transactions

On a copy of the database, a dispense of 6 packs of paracetamol committed and the balance moved from 744 to 738. A dispense larger than the stock was rolled back and the balance stayed 738. A second booking for doctor 2 at 2026-09-03 08:00 failed the unique constraint and was rolled back; the appointment count stayed 428.

Two clerks who read a balance of 734 and wrote back their own arithmetic lost an update: the stored figure

became 731 instead of 726. Two atomic `stock_qty = stock_qty - ?` statements left 726. A second connection could not start a write while the first held `BEGIN IMMEDIATE`; SQLite returned `database is locked`. After the first connection committed, the second write proceeded. Stock and bookings therefore wait for a committed value.

Portfolio 4: other models

Clinical notes are documents. Of 24 notes built from real encounters, three carry an antenatal object and the rest do not have that field. A temperature query returns four febrile notes and ignores notes with no vital signs. The same patients also supply a key-value cache of 47 unpaid balances (ACC-0009 owes NGN 3,605.00), a sparse laboratory sheet keyed by encounter, and eight referral edges from General Outpatient to another department. Patients, stock, and money stay in the relational database.

Portfolio 5: distribution

Patients are sharded by `patient_id`, 60 on each of two shards, because the clinic's working questions are about one patient. Payments stay on that patient's shard: NGN 289,430.00 on shard-0 and NGN 365,355.00 on shard-1. A clinic-wide count has to ask both shards. Each shard has a replica that is updated after the primary. One inserted demonstration patient was visible on the primary (count 61) and missing on the replica (count 60) until the pending write was applied. When the shard-1 primary was marked down, shard-0 still answered and the shard-1 replica could be read for enquiry only. Payments are strongly consistent on the primary. Notes may lag. No cloud account was opened; the log is the executed evidence, and the write-up maps it onto managed PostgreSQL plus a document replica set.

Reflection

The useful constraint was not the number of tables. It was deciding which facts must agree at the moment of a sale. Stock, the stock movement, and the bill belong in one transaction. A clinical note does not, which is why the note is allowed to be a document and, later, an eventually consistent replica. I also found that an index is not an improvement until the plan changes. The stock index and a second booking index would have been easy to add and would not have made those queries faster. The seed can give one adult patient both a paediatric visit and an antenatal visit, because department is chosen separately from age. A production rule would stop that. The sample is large enough to show plans and locks, and too small to justify a real second PostgreSQL server. The shard script is there to make the lag and the partial outage visible, not to pretend the clinic already runs in two regions.

Repository

The work is arranged as `portfolio-1-database-design` through `portfolio-5-distributed-cloud`, with `README.md` at the root. `python run_all.py` rebuilds the database and the evidence. The reviewer's starting point is the GitHub profile https://github.com/Asiya-Ya-u and the repository https://github.com/Asiya-Ya-u/advanced-database-portfolio. The repository should be private, with the reviewer added as a collaborator, before the LMS deadline.

AI-use declaration

I used an AI assistant in Cursor (Grok) to draft the schema, the Python scripts, the diagram, and the wording of these explanations. I did not treat the draft as finished until the scripts had been run. The plan files, the transaction log, the NoSQL results, and the cluster log are the output of those runs. I am responsible for the accuracy of what is submitted and for explaining the design, the queries, and the transaction results if the

work is selected for verification.

References

Elmasri, R. and Navathe, S. B. *Fundamentals of Database Systems*. Pearson.

Silberschatz, A., Korth, H. F. and Sudarshan, S. *Database System Concepts*. McGraw-Hill.

SQLite Consortium. Query planner and transaction documentation. https://www.sqlite.org/docs.html

MongoDB Inc. Data modelling introduction. https://www.mongodb.com/docs/manual/data-modeling/

Brewer, E. "CAP twelve years later: how the rules have changed." *Computer*, 45(2), 2012.

Miva Open University. MIT 8103 continuous assessment brief, September cohort, 2026/2027.